



Spectral Graph Analysis of Bipartite Graphs for Advanced Attack Detection

Majed Jaber, Pierre Parrend, Nicolas Boutry

► To cite this version:

Majed Jaber, Pierre Parrend, Nicolas Boutry. Spectral Graph Analysis of Bipartite Graphs for Advanced Attack Detection. European Interdisciplinary Cybersecurity Conference (EICC25), Apr 2025, Rennes, France. ⟨hal-05038776⟩

HAL Id: hal-05038776

<https://hal.science/hal-05038776v1>

Submitted on 17 Apr 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



HAL Authorization

Spectral Graph Analysis of Bipartite Graphs for Advanced Attack Detection

Majed Jaber^{1,2}[0009–0005–2882–4647], Nicolas Boutry¹[0000–0001–6278–4638], and Pierre Parrend^{1,2}[000–0002–1680–1182]

¹ Laboratoire de Recherche de l'EPITA, 14-16 Rue Voltaire, Le Kremlin-Bicêtre, France

² ICube, UMR 7357, Université de Strasbourg, CNRS, IDEM

Abstract. Spectral graph theory offers powerful tools for understanding graph properties through spectral signatures. This work leverages the inherent link between graph topology and spectral characteristics to enhance anomaly detection in network traffic, particularly in medical IoT networks. We introduce *SPECTRA*, a spectral graph analysis technique designed to detect anomalies in dynamic and complex network structures. This method incorporates five spectral metrics, including the newly proposed BiFlowness metric derived from *Singular Value Decomposition (SVD)*, which captures the flow dynamics within bipartite graph topologies. By combining these spectral metrics, *SPECTRA* provides a comprehensive model for detecting and analyzing advanced cyberattack patterns, such as multistep intrusions, in critical systems. Focusing on hybrid topologies that integrate star and bipartite structures, this technique applies spectral analysis to evolving networks, enabling the detection of attacks (port scanning, fingerprinting) over time. Performed experiments validate the effectiveness of *SPECTRA* across IoT datasets, demonstrating its superiority in identifying attack behaviors. The proposed approach aligns with the critical demands of medical IoT environments by providing a good threat detection procedure to enhance security in sensitive networks.

Keywords: Spectrum · Spectral graph · Bipartite topology · Attack detection · Laplacian matrix. · Singular Value Decomposition

1 Introduction

In recent years, medical networks have emerged as critical targets for cyberattacks due to the sensitivity of the data they handle. These networks, often comprised of interconnected devices, servers, and systems, face sophisticated and multi-step attacks that exploit vulnerabilities in their structure. A multi-step attack is a cyberattack that involves a series of interconnected and sequential stages, each designed to achieve specific objectives, such as gaining access, followed by privilege elevation, and finally data theft or disruption. Defending against these advanced attacks requires understanding their behavior to capture the evolving connectivity patterns within these networks. Through spectral graph analysis, it is possible to uncover the spectral properties of the networks over time, where spectral properties are important in capturing dynamic and evolving relationships between the devices in the medical networks. To address the complexity of these hybrid topologies, the contribution of this work is the introducing of a new metric called *BiFlowness* derived from singular values obtained through *Singular Value Decomposition (SVD)*. This metric is specifically designed to measure the flow within bipartite topologies, capturing critical aspects of network connectivity and behavior. The highlights of this paper are as follows: (1) we introduce a *BiFlowness* metric based on singular values; (2) *BiFlowness* metric is designed to measure the flow within the bipartite subgraph of the network; (3) we extend the graph conception beyond star topologies to a combination of bipartite and star graphs; (4) we adapt the previous methodology *SPECTRA* [JBP24] to handle these hybrid topologies; and (5) we provide extensive benchmarks demonstrating the effectiveness of combining the five metrics for analyzing and understanding hybrid network topologies.

2 State-of-the-Art

Advancements in graph-based methodologies, spectral analysis, and machine learning have significantly contributed to cybersecurity, particularly in detecting and mitigating sophisticated cyberattacks. Bipartite graph

approximation methods enable efficient graph representations to address various challenges, including optimizing graph partitioning and clustering in large-scale networks. Spectral graph analysis provides powerful tools for attack detection by leveraging eigenvalues and eigenvectors to uncover hidden patterns and anomalies in network behavior. Meanwhile, machine learning and deep learning approaches, including graph-based techniques, offer robust solutions for anomaly detection, addressing the limitations of traditional statistical methods. These developments collectively enhance the ability to analyze complex network structures and detect threats, forming the foundation for modern cybersecurity frameworks.

Bipartite Graph Approximation Methods: Numerous methods have been developed to approximate graphs into bipartite structures, each addressing challenges and providing significant contributions to this field. The Bipartite Polarization Problem, as introduced in [AABM22], generalizes the maxcut problem by seeking the most polarized bipartition in a labeled graph. This problem is particularly relevant for instance to social networks analysis like in the Reddit case, where debates are modeled as graphs. The authors propose an integer programming-based exact algorithm and local search algorithm variants. The improved local search approach delivers approximate solutions that closely match the exact method’s optimal results, while significantly reducing computational time. These algorithms balance user sentiment and interaction polarity across the bipartition, providing a scalable framework for social network analysis. An ADMM-based bipartite graph approximation algorithm is proposed in [JWT⁺19]. This method uses the Alternating Direction Method of Multipliers (ADMM) to derive a real adjacency matrix that closely resembles the original graph, while maintaining bipartite properties. The ADMM framework ensures efficient optimization, making this algorithm particularly effective for applications requiring scalable and accurate bipartite graph approximations. The One-Step Bipartite Graph Cut (OBCut) algorithm, discussed in [FWL23] and [ZCO17], offers an efficient approach to graph bipartitioning. OBCut introduces a normalized bipartite graph cut criterion that balances node size and edge volume. By integrating adaptive anchor learning and bipartite graph partitioning into a unified optimization framework, OBCut achieves effective and scalable partitioning. The alternating optimization algorithm employed in this method ensures linear complexity relative to sample size and improves clustering performance across various datasets. In [LL23], the authors address the problem of finding a maximum induced bipartite subgraph by identifying the largest subset of vertices that induces a bipartite subgraph. Although this problem is NP-hard in general, the authors propose polynomial-time algorithms for specific graph families, such as circular-arc graphs and i -triangulated graphs. For instance, an $O(n^8)$ algorithm is proposed for circular-arc graphs, providing efficient solutions for specific constrained cases.

Spectral Graph Analysis for attack detection: Several spectral analysis approaches have been applied in cybersecurity. For example, power spectral density (PSD) was used in [CKT02] to detect periodic changes in Transmission Control Protocol (TCP) transport, aiding in defending against DoS attacks. Another approach introduces an adversarial attack on Graph Convolutional Networks (GCNs) by perturbing graph structures to maximize spectral distance. This disrupts spectral filters, compromising GCN performance in both test-time and training-time scenarios. Methods such as MC-GPCA and MC-GDL in [CCH16] reduce graph complexity and perform dictionary learning, utilizing spectral decomposition through Singular Value Decomposition (SVD). Similarly, [YWB10] analyzes adjacency matrix eigenvalues and eigenvectors to detect cybersecurity attacks by identifying attacker patterns in spectral space. Techniques like discrete Fourier transform (DFT) combined with hypothesis testing [CH06] and dynamic graph modeling [BY06] further expand anomaly detection and pattern discovery in static and evolving networks.

Machine Learning for anomaly detection: Anomaly detection methods are categorized into statistical and machine learning (ML)-based approaches. Statistical methods like ReTiNA [NA18] and probability-based frameworks [HRD16] offer real-time and scalable anomaly detection but face accuracy limitations [HGP20]. ML-based platforms, such as CAMLPAD [HGP20], use diverse data sources (e.g., logs and network traffic) and algorithms like Logistic Regression, Decision Trees, and k -Means to improve detection. However, issues such as limited labeled data [BH21], skewed datasets, label noise, and rapidly evolving cyber threats remain significant. Graph-based ML methods, such as random walks [PARS14, GL16] and StrGNN [CCL⁺21], address these challenges by leveraging graph structures for anomaly detection. Deep learning techniques,

including Graph Convolutional Networks (GCNs) [KW16], GNN model for analyzing malicious web requests [LZL⁺23], HyperGCN[YNY⁺18], and Graph Attention Networks (GATs)[VCC⁺17], further enhance detection by capturing complex relationships in graph data. Weakly supervised frameworks [LCQ⁺21] expand these capabilities by overcoming data limitations and improving anomaly detection in dynamic network environments.

3 Network Graph Analysis: Spectral and Structural Perspectives

This section explores network graph analysis from both spectral and structural perspectives [MW19]. The structural approach focuses on matrices that are the adjacency, degree, and Laplacian matrices, which encode essential topological information [Kun15]. The spectral perspective leverages eigenvalues and eigenvectors to extract deeper insights into graph connectivity, bipartiteness, and partitioning strategies. By analyzing the Laplacian eigenvalues, we uncover fundamental network properties, in specific one of the important properties we gather is the inter connectivity between clusters and components of the graph. Furthermore, we examine bipartite graph structures, using eigenvector-based partitioning techniques to reveal hidden patterns in network traffic. The following subsections detail these concepts, starting with graph-based network representation, then structural matrices, followed by spectral properties of graphs, eigenvalue analysis, and bipartite graph partitioning. These techniques collectively enable a more comprehensive understanding of network behavior for cybersecurity applications.

3.1 Graph-Based Network Representation for Cybersecurity

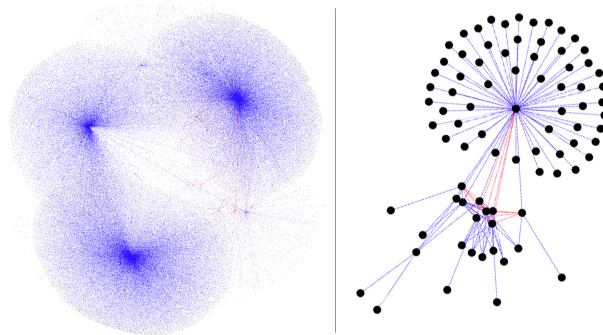


Fig.1: Graph representations show the TonIot and Botnet datasets respectively from the left to the right.

Graph theory is a crucial tool in cybersecurity, enabling the modeling of networks to detect threats and assess vulnerabilities [For07]. By utilizing graph representations 1, particularly adjacency and Laplacian matrices, network behaviors can be evaluated, and anomalies such as abnormal behaviors can be identified. A network is composed of interconnected *nodes*, such as computers, servers, switches, and routers, which facilitate data exchange and shared resource access [Lóp07]. Essential network characteristics include *inter-connectivity*, which enables wired or wireless connections; *data exchange*, which ensures the transfer of files and messages; *shared resources*, such as printers and internet access; and compliance with *protocols* that regulate communication between devices [Sta03]. Networks can be structured as graphs, where nodes represent devices, and edges signify interactions such as data transmission, communication protocols, or connection rates.

3.2 Structural Matrices in Graph Theory

Let G be a graph consisting of n nodes. Its structure is defined by three fundamental matrices. The first is the *Adjacency Matrix* A , an $n \times n$ matrix where each element A_{ij} is 1 if nodes i and j are connected, and 0 otherwise. The second matrix, the *Degree Matrix* D of G , is a diagonal $n \times n$ matrix where each diagonal element D_{ii} is given by $\sum_{j=1}^n A_{ij}$, while all off-diagonal elements are set to 0. Finally, the *Laplacian Matrix* L is defined as $L = D - A$ [Mer94]. This matrix plays a crucial role in spectral graph theory, capturing both the spectral properties and structural integrity of the graph.

3.3 Laplacian Eigenvalues: The Graph's Spectrum

When a network is modeled as a graph, its topological characteristics can be derived through spectral graph analysis [Chu97] by examining the Laplacian eigenvalues. Various properties emerge from this analysis, such as the number of eigenvalues equal to zero, which corresponds to the count of connected components in the graph, and the largest eigenvalue, which reflect the highest degrees between the inter-connected nodes [BJ09]. Additionally, the robustness of this spectral representation to edge rewiring has been demonstrated, meaning that *small alterations in graph weights result in minor variations in the graph spectrum* [dLdRVDH14].

Let Λ be the spectrum of the Laplacian matrix L , Z is equal to the number of eigenvalues equal to 0 in Λ . We recall that, in linear algebra, any eigenvalue λ of a square matrix $\mathbf{A}$, satisfies the relation $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ [BC⁺06], with $\mathbf{v}$ the eigenvector associated to the eigenvalue λ . The Laplacian matrix's spectrum is important as it uncovers critical structural properties of graphs such as the number of components of the graph (sub-networks) and the connectivity of the overall network structure, maintaining its relevance even when the graph's topology is modified [BC⁺06], thereby emphasizing its utility in exploring and understanding the convolutions of networks.

4 Introducing Spectral Metrics : Indicators

In our research, we explore the Laplacian matrix spectrum and how its eigenvalues help reveal different graph structures. These values act as subtle clues to detect specific cyberattacks. By analyzing datasets, we find that star graphs and bipartite graphs appear frequently, making them the focus of our experiments. We study normal and abnormal network behaviors over these topologies, tracking spectral changes to spot unusual activity. We define five spectral metrics, the first four metrics are based on the eigenvalues of L while the last one rely on the Σ decomposed from L' after applying *SVD*, showing that they effectively identify and analyze advanced network attack patterns. The five metrics are defined as follows:

In this context, $\Lambda^{(t)}$ represents the spectrum of L at time t , and for each $i \in [1, n]$, $\lambda_i^{(t)}$ denotes the i^{th} smallest eigenvalue within $\Lambda^{(t)}$. And $\Sigma^{(t)}$ represents the set of singular values of L' at time t sorted in increasing order, and for each $j \in [1, n]$, $\sigma_j^{(t)}$ denotes the j^{th} smallest eigenvalue within $\Sigma^{(t)}$.

4.1 Spectral Metrics for Network Analysis

In this subsection, we present the first four spectral metrics, which were previously introduced in the published work [JBP24]. These metrics quantify different aspects of network connectivity and structural changes, aiding in the detection of network intrusions and anomalies.

Connectedness The first metric, *Connectedness*, represented as $\mu_1(t)$, measures the overall connectivity within the network. It is formulated as follows:

$$\mu_1(t) = \frac{\exp \frac{1}{Z(t)}}{\exp(1)}$$

where $Z(t)$ represents the count of zero values in the network's spectrum. This metric helps detect changes in network topology. When μ_1 approaches 1, it may indicate an intrusion, as it suggests the formation of new links between previously disconnected components. As $Z(t)$ increases towards $+\infty$, μ_1 converges to $\exp^{-1} \approx 0.367$. Conversely, when $Z(t)$ nears 1, μ_1 tends toward 1.

Flooding The second metric, *Flooding*, denoted as $\mu_2(t)$, assesses the network's overall connectivity by analyzing the algebraic connectivity of each connected component. It is defined as follows:

$$\mu_2(t) = \left(\frac{1}{\mathcal{N}} \sum_{i=\mathcal{Z}(t)+1}^{\mathcal{Z}(t)+\mathcal{N}} \lambda_i^{(t)} \right) - 1$$

where $\mathcal{N}$ represents the number of central network devices, such as switches and servers. This metric accounts for both the existence of node connections and their associated weights, such as packet count or byte size. A higher number of connections with substantial weights leads to an increase in μ_2 .

Wiriness The third metric, *Wiriness*, represented as $\mu_3(t)$, evaluates the density and strength of network interconnections by considering edge weights. It is given by:

$$\mu_3(t) = \frac{1}{\mathcal{N}} \sum_{i=n-\mathcal{N}+1}^n \lambda_i^{(t)}$$

This metric captures the impact of new connections on the network's structure. It increases with higher packet sizes and a greater number of network requests, making it useful for identifying network flooding or large-scale data transfers.

Asymmetry The fourth metric, *Asymmetry*, represented as $\mu_4(t)$, quantifies variations in spectral properties across the network:

$$\mu_4(t) = \text{Cardinality} \left(\left\{ i \in [2, n] ; \lambda_i^{(t)} - \lambda_{i-1}^{(t)} > \varepsilon \right\} \right)$$

where $\varepsilon = 10^{-12}$ is included to reduce approximation errors. This metric evaluates the smoothness of the spectral curve. When the network maintains similar structural patterns, μ_4 remains low. However, an increase in μ_4 suggests changes in connectivity patterns, potentially signaling a security threat.

4.2 Introducing BiFlowness metric

Eigenvector-Based Bipartite Graph Partitioning and Analysis Partitioning graphs into known topologies is essential for understanding network connectivity and identifying structural patterns such as star graphs, and particularly in bipartite graphs [ZHD⁺01]. Spectral graph analysis provides a mathematical framework for achieving this, leveraging eigenvectors of the Laplacian matrix to extract sets of nodes efficiently by partitioning. In the context of cybersecurity, we delve through the importance of bipartite topologies in cybersecurity network analysis and we propose our methodology in the upcoming sections.

Different types of attacks, such as port scanning and OS fingerprinting, display behaviors that results in topological patterns resembling bipartite structures. Within the framework of bipartite graph topologies, we adopt a detailed methodological approach. A bipartite graph is composed of two disjoint sets of nodes, where edges connect only nodes from different sets. Identifying these partitions using spectral methods relies on the eigenvalues and eigenvectors of the graph's Laplacian matrix. The *Fiedler vector*, which corresponds to the eigenvector associated with the second-smallest eigenvalue of the Laplacian matrix, is widely used for spectral graph partitioning as it provides an optimal way to divide a graph into two balanced components [DMM22]. Another key eigenvector in spectral analysis denoted by $\mathbf{EV}_n$ is associated with the maximum eigenvalue of the Laplacian matrix, which plays a significant role in estimating bipartite sets in our methodology, which is detailed in the next section.

Bipartite graph estimation and partitioning In order to apply spectral graph analysis to graph topologies, the first step is to detect bipartite graphs and estimate their partitions (sets) in network traffic data. The bipartite partitioning process leverages the eigenvectors of the Laplacian matrix L , specifically starting with the eigenvector corresponding to the second-smallest eigenvalue of L [DMM22] and subsequently considering the eigenvectors of the third-smallest, fourth-smallest eigenvalues, and so on. The partitioning is determined

by the sign of each entry in the eigenvector $\mathbf{v}_i$, where positive and negative values define the two groups. As illustrated in Fig. 2, after exploring various scenarios on $G_{m,n}$, our methodology selects two eigenvectors for partitioning: the Fiedler vector (associated with the second-smallest eigenvalue) and the eigenvector corresponding to λ_{max} . To evaluate the effectiveness of both partitioning methods, we assess their results against the actual partitions using two clustering metrics: the **Adjusted Rand Index (ARI)** [CR23] that measures the similarity between predicted and true partitions, accounting for chance and **Normalized Mutual Information (NMI)** [SGB22] that evaluates the amount of shared information between the partitions, scaled to ensure comparability. Figure 2 illustrates the initialization with eight nodes, where a bipartite graph evolves by incrementally adding one edge at each step, continuing until a complete bipartite graph is formed at evolution step fifteen. At each step, partitioning is performed using both methods, and the ARI and NMI scores are computed to evaluate the clustering performance. The bottom right of Fig. 2 presents line graphs of these clustering evaluation metrics, demonstrating that the eigenvector of λ_{max} yields superior clustering results in bipartite graph partitioning. Notably, from evolution step 9 onward, when each node has at least one edge, this approach aligns more closely with the actual partitions (according to both *ARI* and *NMI* metrics) compared to the Fiedler vector.

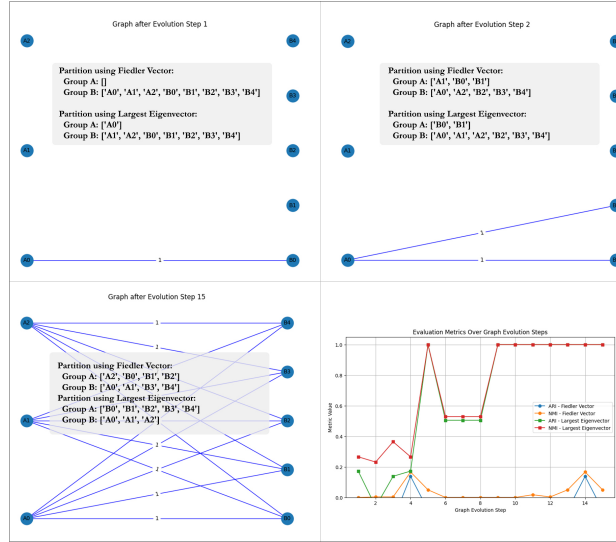


Fig. 2: Graph evolutions representing the procedure used to partition bipartite graph at each step, with comparison metrics.

Sub-matrix extraction Let $\text{card}(X)$ and $\text{card}(Y)$ represent the cardinalities of the two sets of X and Y of $G_{m,n}$ respectively after partitioning based on the eigenvector of λ_{max} . The objective is to extract a submatrix of L , denoted as L' , which encapsulates the interconnections between the two sets X and Y .

Consider a bipartite graph $G_{m,n} = (U, V, E)$ consisting of two disjoint node sets, where $X = \{x_1, x_2, \dots, x_m\}$ and $Y = \{y_1, y_2, \dots, y_n\}$, and where the edge set E contains only inter-set edges, i.e., edges exclusively between X and Y , with no intra-set connections. The adjacency matrix A of this bipartite graph takes the form:

$$A = \begin{bmatrix} 0 & B \\ B^T & 0 \end{bmatrix}$$

where B is an $m \times n$ incidence matrix such that $B_{ij} = 1$ if x_i is connected to y_j , and B^T represents the transpose of B . The other blocks contain only zeros, ensuring no intra-set edges exist within the same group. A submatrix of A , denoted as $A' = B$, retains information about the edges between the two groups.

Consequently, L' is derived from L by extracting its last $\text{card}(Y)$ columns and first $\text{card}(X)$ rows, preserving the spectral, weight distribution and connectivity properties.

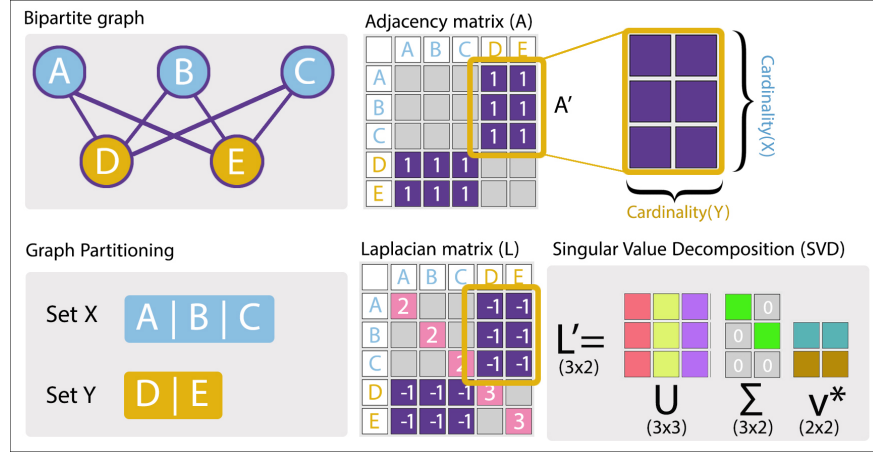


Fig. 3: The figure illustrates the extraction of L' from $G_{3,2}$. At the bottom left, the two sets X and Y are obtained after partitioning, as described in Section 4.2. A detailed depiction of the extraction process for the matrices A' and L' is shown at the top right and bottom middle, respectively, where SVD is applied to L' to compute the singular values Σ .

An example is provided in Fig. 3, depicting a single evolution window in our methodology. At this stage of evolution, a bipartite graph $K_{3,2}$ is constructed, representing the traffic logs over the network at tw_i . On the right side of the graph, the adjacency matrix A is displayed, with a focused view on its submatrix $A' = B$, which has dimensions $\text{card}(X) \times \text{card}(Y)$. Similarly, the submatrix L' of L is shown, capturing the connectivity structure between the two sets. In the next section (Sec. 4.2), we delve further into singular value decomposition (SVD) and its relevance in *SPECTRA* methodology.

Sigma from singular value decomposition (SVD) Here is the last step of analyzing the bipartite topologies, in this step SVD decomposition [Abd07] is applied over submatrix L' as shown in Fig. 3:

$$\text{Sub-Matrix } L'_{m \times n} = \begin{matrix} U \\ [u_1 \ u_2 \ \dots \ u_m] \end{matrix}_{m \times m} \begin{matrix} \Sigma \\ \begin{bmatrix} \sigma_1 & 0 & \dots & 0 \\ 0 & \sigma_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \sigma_r \end{bmatrix} \end{matrix}_{m \times n} \begin{matrix} V^T \\ \begin{bmatrix} v_1^T \\ v_2^T \\ \vdots \\ v_n^T \end{bmatrix} \end{matrix}_{n \times n}$$

where $\mathbf{u}_i$ are the left singular vectors (columns of U), $\mathbf{v}_i$ are the right singular vectors (rows of V^T), and σ_i are the singular values in decreasing order. An important observation is that most of the information regarding the graph's topological structure is contained within the set Σ [Dax13]. Notably, Σ singular values reflect the eigenvalues of L , which motivated us to further explore this connection. Consequently, we developed a bipartite spectral metric that leverages the values of Σ , which will be discussed in detail in section 4.

BiFlowness Here we introduce a new spectral metric, *BiFlowness* denoted as $\mu_5(t)$. *BiFlowness* is a novel contribution that extends spectral analysis by incorporating singular value decomposition (SVD), and since singular values are always non-negative, our metric is guaranteed to be positive. It is defined as follows:

$$\mu_5(t) = \sum_{j=2}^n \sigma_j$$

where σ_j represents the singular values obtained from the decomposition of the sub-matrix L' of L . This metric provides insights into intra-group connections by analyzing eigenvalue structures indirectly. By capturing interconnections between bipartite network groups, μ_5 enhances the understanding of network structural changes over time. It is particularly useful for identifying coordinated attacks that exploit internal network structures. This additional metric strengthens network intrusion detection by complementing the existing four spectral metrics, enabling a more comprehensive evaluation of network behavior especially the interconnections of the two sets of the bipartite topologies.

5 Methodologies and Techniques

In this section, we present a set of methodologies designed to enhance network security analysis. Beginning with fundamental dataset examination and progressing to advanced spectral-time-window classification (with different combination of spectral metrics), each approach incrementally deepens the understanding of network behavior and threat detection. These methods establish a framework for identifying and analyzing anomalies in network traffic, excluding time and sequence data to reduce bias and improve detection accuracy.

5.1 Methodological Approaches

Next, we will present the methods we used to build our approaches, along with their respective methodologies, beginning with the *OD*, then progressing to *STS* and *STW*, and ending up with *SPECTRA*.

Original data-logs (*OD*) In this approach, which serves as a baseline for further evaluations, we use datasets without any preprocessing. All dataset features are included, except those related to date, time, and packet arrival sequences. These exclusions prevent bias, as many attacks tend to occur at the same timestamps. The main goal is to establish a reference point for comparing other methods [DM17][SLV⁺20]. Our approach consists of two key steps:

- Using the dataset without applying any preprocessing steps.
- Retaining all dataset features except sequence and time-related attributes.

Table 1: Aggregation methods for dataset features

Feature	Aggregation	Feature	Aggregation
Total packets	Sum	Source packets	Sum
Destination packets	Sum	Total bytes	Sum
Source bytes	Sum	Destination bytes	Sum
Sum	Mean	Label	First
Total rate	Mean	Source rate	Mean
Destination rate	Mean	Duration	Mean
Mean	Mean	Sum	Mean
Max	Mean	Min	Mean

Time-series for spectral analysis (*STS*) This approach extends the previous one by incorporating all dataset features while continuing to exclude time and sequence attributes. The aim behind this approach arises from the observed imbalance between attack and normal traffic within the dataset’s log entries. Furthermore, time series analysis offers key advantages in managing dynamic graphs that evolve over time [PMRC21]. In this context, each unique timestamp in the dataset corresponds to a specific stage in the

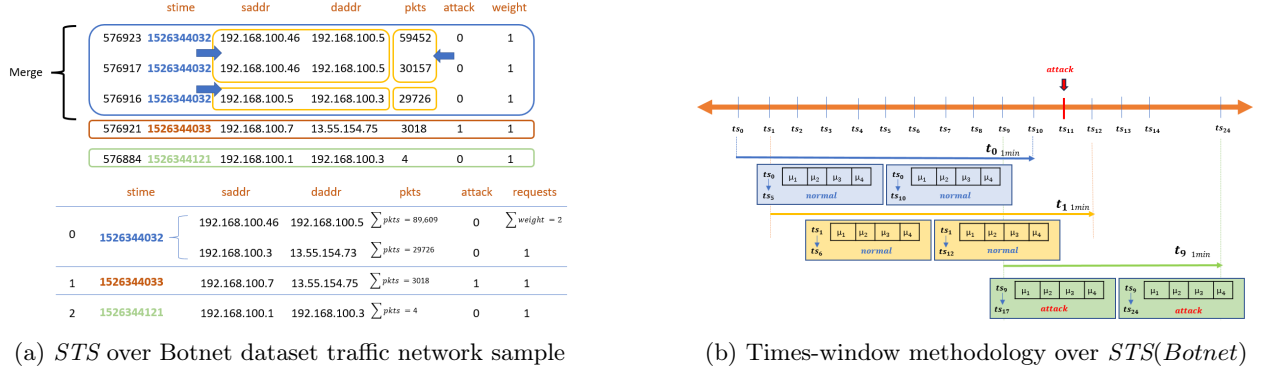


Fig. 4: Time-series and time-window methodologies over Botnet data sample

graph's evolution. To implement this approach, we follow the methodology illustrated in Fig.4a, where data is aggregated based on predefined constraints and grouping criteria. First, we define a time constraint t ($t \geq 1\text{sec}$) to establish each stage of graph evolution. Then, we iterate through the log set at specific time intervals t , represented as t_s .

Next, traffic logs are grouped based on specific conditions: logs must originate from the same source IP, be directed to the same destination IP, and share the same traffic type (e.g., normal). This grouping strategy is a core component of our approach. If such traffic occurs within the same time window t , it is aggregated. The aggregation process is feature-dependent, with each feature assigned a specific aggregation method. By following this procedure, we transform the dataset into a time-series representation, denoted as $STS_{dataset}$, ensuring well-defined constraints on both time and feature selection.

Time-windowing for spectral analysis (STW) In this approach, we further explore the representation of topological structure graphs within time-series data. To refine the time range in which attacks occur, we introduce a time-windowing technique [TBSK18,Bal22]. This method employs a sliding window that moves across the time-series dataset $STS_{dataset}$, analyzing both quantitative and topological features within each window. By doing so, our methodology adapts to variations in the network's topological structure, enhancing its ability to detect dynamic changes. Specifically, the sliding window operates under predefined constraints within the time-series data. These constraints include the time range of the sliding window, denoted as tw , and the minimum number of data logs required within each time window, represented as MIN_{tw} , where $MIN_{tw} \geq 2$. Next, we examine the processing steps that take place within each sliding window at time t_i .

As illustrated in Fig.4b, during each time window tw , the extraction process focuses solely on quantitative features, intentionally omitting the topological structure. This aspect will be explored in the next approach, discussed in Section 5.1, alongside our spectral metrics. The extracted features within tw include packet, byte, and rate-related attributes. These features are already present in $STS_{dataset}$, while other attributes are deliberately excluded. This exclusion serves a strategic purpose: to enable a direct comparison with the spectral approach and to emphasize the effectiveness of spectral metrics that rely on these features, without incorporating additional attributes at this stage.

It is important to note that as the sliding window moves through the time-series, it may encompass two or more timestamps (data rows). The goal is to construct a new dataset where each data row represents an aggregation of the data rows within a single sliding window. This aggregation process follows the methods specified in Table 1 and assigns labels based on the presence or absence of attacks within the window. If an attack is detected within a given window, the aggregated row is labeled accordingly; otherwise, it is classified as normal. Once the aggregation is complete, the newly generated data-frames, denoted as STW_{bot} and STW_{ton} , are obtained. These data-frames correspond to the number of sliding windows within the time-series for the Botnet-IoT and Ton-IoT datasets, respectively.

Spectral methodology applied on time-windowing (*SPECTRA*) In this method, we leverage graph structures within each designated time-window. While this approach shares similarities with the previous *STW* method, it introduces a key difference in the handling of the time-window.

Step 1 Data rows are transformed into a graph and then processed within two distinct sub-time-windows, Sub_{TW_1} and Sub_{TW_2} . Here, Sub_{TW_1} represents the graph constructed from TW_0 to $TW_{n/2}$, while Sub_{TW_2} extends from TW_0 to TW_n , where n denotes the total number of timestamps within the time-window. The division of a time-window into these two sub-windows allows for tracking variations over time, which directly influences the computed metrics.

Step 2 Once graphs are formed over the sub-time-windows, we extract the Laplacian matrix L for Sub_{TW_1} and Sub_{TW_2} , producing L_1 and L_2 . Next, we compute the spectra Λ_1 and Λ_2 for each Laplacian matrix. The resulting datasets, $SPECTRA_{bot}$ and $SPECTRA_{ton}$, correspond to the number of sliding windows across the time-series for the Botnet-IoT and Ton-IoT datasets, respectively. With these datasets prepared, we proceed to the application of various classification techniques.

6 Implementation

To implement this methodology and approaches, we employed two Python notebooks, both using *Original Data-logs(OD)*, *Time-series for Spectral Analysis(STS)*, *Time-windowing for Spectral Analysis(STW)*, and *Spectral Methodology Applied on Time-windowing(SPECTRA)* in their approaches. The two notebooks are applied to both botnet-IoT and Ton-IoT datasets. To foster reproducibility and contribute to advancing research in cybersecurity, we have made the source code for the metrics and their evaluation available on GitHub³. This reproducible capsule includes the necessary scripts for replicating the experiments. These notebooks mainly focus on evaluating μ_5 approaches in comparison with the *OD* baseline approach. For previous work, in comparison with the *STS* and *STW* approaches you can find them here in GitHub⁴. Additionally, as part of the LRE research team, we have developed an open-source GPML library⁵, which includes various strategies used for spectral graph analysis.

6.1 Experiments: Practical Applications

This section discusses the applications and case studies conducted as part of our research, building upon the work published in [JBP24]. The focus is on the Botnet-IoT and Ton-IoT datasets 1. The experiments progress through multiple stages, starting with the application of the *OD* methodology as a baseline, followed by the *SPECTRA* methodology incorporating various spectral metrics we introduced. Within *SPECTRA*, traffic logs undergo *STS* and *STW* analysis and processing. Each subsection details the methodology employed. These experiments aim to evaluate the effectiveness of our approaches in detecting network attacks within IoT environments.

Application of *OD* For the Botnet-IoT and Ton-IoT datasets, applying the *OD* methodology follows a straightforward process. Initially, all numerical features from both datasets were considered. Then, a subset of these features was selected, excluding certain ones as specified in Table 2 for Botnet-IoT and Table 3 for Ton-IoT. The selected features are those commonly found in the literature across different datasets and attributes present in network traffic. Time-related features (such as stime and ltime) were removed to prevent biased results, along with sequence-based features (such as seq and pkseqId). Additional features specific to the Botnet-IoT dataset, as referenced in [KMST19], were also excluded, as they are incorporated during preprocessing and are not typically found in raw traffic data. To enhance the model’s generalizability, only attributes shared across multiple datasets or present in network traffic were retained. This process resulted in new datasets with the same number of logs but with selected features, denoted as OD_{ton} for Ton-IoT and OD_{bot} for Botnet-IoT.

³ <https://github.com/majedjaber1996/jaber.2025.eicc>

⁴ <https://github.com/lre-security-systems-team/jaber.2024.ares.git>

⁵ <https://gitlab.cri.epita.fr/laboratoires/lse/research-devs/gpml>

Table 2: Extracted and eliminated features of Bot-IoT dataset

Features Extracted	Features Eliminated
flgs_number	pkSeqID
dur	seq
proto_number	stime
pkts	ltime
bytes	TnBPSrcIP
state_number	TnBPDstIP
mean	TnP_PerProto
stddev	TnP_Per_Dport
sum	AR_P_Proto_P_SrcIP
min	AR_P_Proto_P_DstIP
max	N_IN_Conn_P_DstIP
spkts	N_IN_Conn_P_SrcIP
dpkts	AR_P_Proto_P_Sport
sbytes	AR_P_Proto_P_Dport
dbytes	Pkts_F_State_F_Protocol_P_DestIP
rate	Pkts_P_State_P_Protocol_P_SrcIP
srate	
drate	

Table 3: Extracted and eliminated features of Ton-IoT dataset

Features Extracted	Features Eliminated
duration	ts
src_bytes	ltime
dst_bytes	
src_pkts	
dst_pkts	
label	
type	

Application of *STS* For the Botnet and TonIoT datasets, the *STS* method builds upon the *OD* approach by incorporating a time-based aggregation. In *STS*, a time window of $t = 1s$ is applied to both OD_{ton} and OD_{bot} . Within this interval, features are aggregated as outlined in Table 1. This process restructures the datasets into a timeseries format, labeled as STS_{bot} for Botnet and STS_{ton} for TonIoT. As a result, the number of data rows typically decreases, aiding in subsequent data analysis and evaluation.

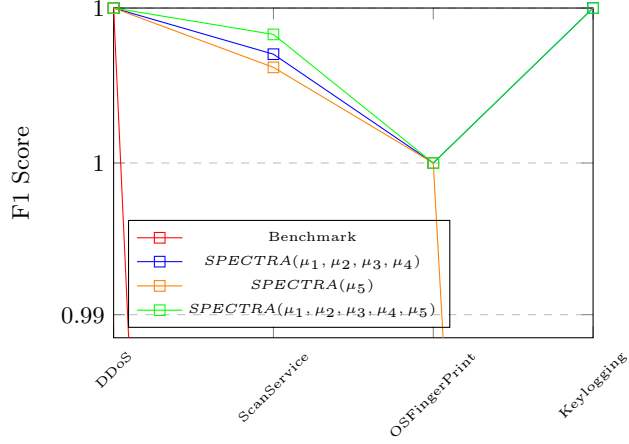
Application of *STW* To enhance data aggregation, the *STW* method is applied alongside the *STS* approach for the Botnet-IoT and Ton-IoT datasets. In *STW*, different time-window ranges are defined: $tw = 1min$ for STS_{bot} with a minimum threshold of $MIN_{tw} \geq 10$, and $tw = 30sec$ for STS_{ton} with $MIN_{tw} \geq 2$. During each t_w , features are aggregated according to Table 1. This process transforms STS_{bot} and STS_{ton} into a new dataset format, denoted as STW_{bot} and STW_{ton} , respectively. In this format, each dataset row represents the aggregated data within the specified tw .

Application of *SPECTRA* Applying *SPECTRA* requires additional processing for each time window T_i . While *SPECTRA* itself does not impose strict constraints, constructing weighted edges in the network graph depends on selecting a specific feature. For both the Ton-IoT and Botnet-IoT datasets, packet count, byte size, and packet rate were chosen as the criteria for edge weighting across three different topologies within the time window. The first topology is weighted by the number of packets transferred, the second by byte size, and the third by packet rate. This selection is based on the observation that these attributes are key indicators for detecting various attack types. For each time window, three sets of ten spectral metrics are computed, corresponding to the three topologies. Metrics from μ_1 to μ_5 are calculated at two intervals: tw (the middle of the time window) and $\frac{tw}{2}$ (the end of the time window). This process generates two datasets, $SPECTRA_{bot}$ and $SPECTRA_{ton}$, where each row consists of spectral metric vectors along with their associated labels.

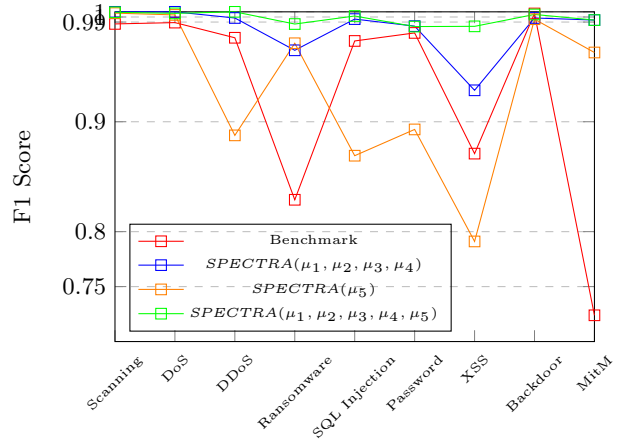
7 Evaluations: Assessing Techniques and Results

In this section, we conduct a detailed evaluation of machine learning performance metrics, including the F1-Score, Balanced Accuracy, and Matthews Correlation Coefficient (MCC), for the XGBoost classifier [CG16]. The assessment covers XGBoost’s performance across different methodologies, specifically the *Baseline* approach(*OD*) methodology, and the *Spectral Methodology Applied on Time-windowing*(*SPECTRA*) methodological approaches, where various spectral metric combinations are analyzed. In the latest paper [JBP24], an evaluation of the *SPECTRA* approach using the first four spectral metrics, from μ_1 to μ_4 , which demonstrated strong performance compared to the baseline, *Time-series for Spectral Analysis*(*STS*), and *Time-windowing for Spectral Analysis*(*STW*) approaches. In this evaluation, we extend our analysis for different metrics combinations of *SPECTRA* analysis. The first incorporates the default features along with only the μ_5 spectral

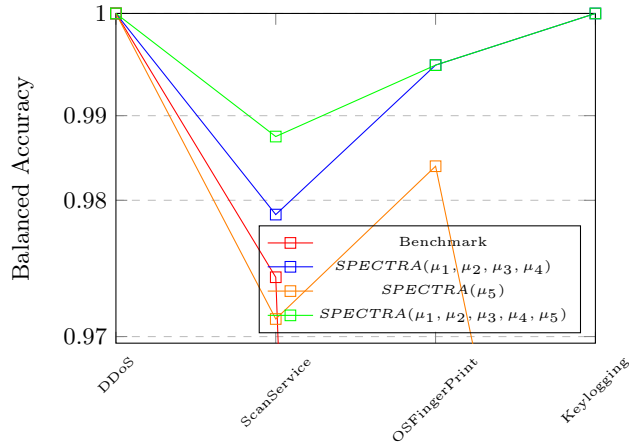
metric, while the second includes spectral metrics from μ_1 to μ_4 , and the last includes all spectral metrics from μ_1 to μ_5 . Our discussion will analyze these different combination strategies and their scores, providing insights into the effectiveness and efficiency of XGBoost across these methodologies.



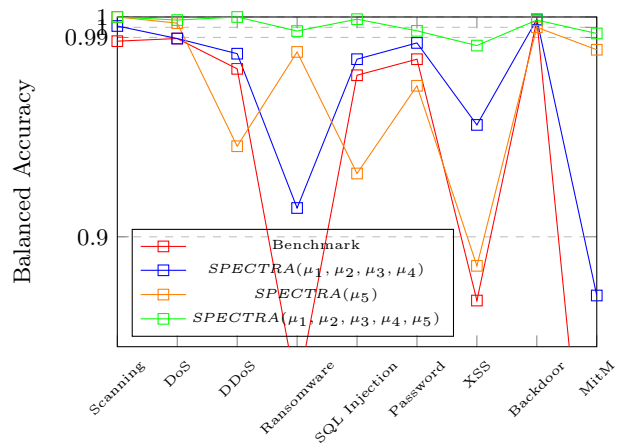
(a) Botnet-IoT Dataset



(b) Ton-IoT Dataset

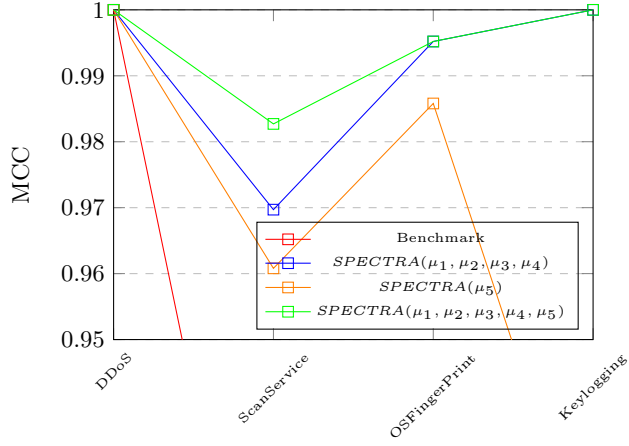


(a) Botnet-IoT Dataset

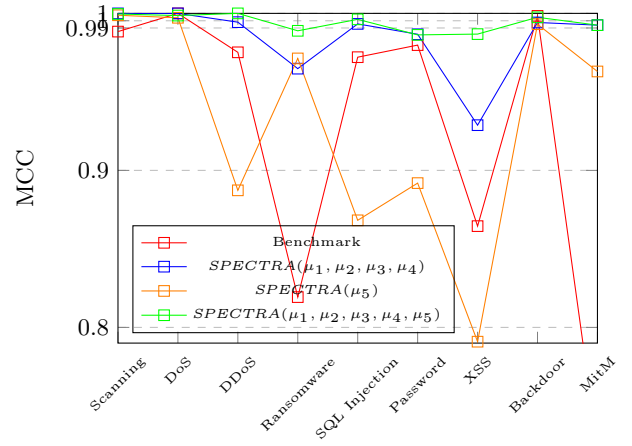


(b) Ton-IoT Dataset

Performance Analysis on Botnet-IoT Dataset: Evaluating the XGBoost classifier’s performance on the Botnet-IoT dataset across different approaches reveals substantial variations in classification effectiveness. The Benchmark method achieves a perfect F1-Score of 1.0 for DDoS, but struggles with OS Fingerprint and Keylogging, where F1-Scores drop significantly to 0.2617 and 0.5333, respectively (Fig. 5a). Introducing the *SPECTRA* methodology with different spectral metric combinations significantly enhances classification performance. $SPECTRA(\mu_1, \mu_2, \mu_3, \mu_4)$ achieves an F1-Score of 0.9986 for ScanService, 0.9953 for OS Fingerprint, and 1.0 for Keylogging. Further improvement is observed with $SPECTRA(\mu_1, \mu_2, \mu_3, \mu_4, \mu_5)$, which outperforms all other approaches across all attack types, achieving an F1-Score of 0.9992 for ScanService and maintaining the highest scores for OS Fingerprint and Keylogging (Fig. 5a). Balanced Accuracy and MCC results confirm these trends. The Benchmark approach struggles with OS Fingerprint, achieving a Balanced



(a) Botnet-IoT Dataset



(b) Ton-IoT Dataset

Accuracy of only 0.5804, while $SPECTRA(\mu_1, \mu_2, \mu_3, \mu_4, \mu_5)$ improves this metric to 0.9953 (Fig. 6a). Similarly, the MCC for OS FingerPrint is lowest in the Benchmark approach (0.3198) but reaches 0.9952 with $SPECTRA$ using all spectral metrics (Fig. 7a). These results highlight that incorporating all five spectral metrics optimally refines the classifier's predictive capability, ensuring superior attack detection.

Performance Analysis on Ton-IoT Dataset: Analyzing the Ton-IoT dataset reveals that the XGBoost classifier's performance varies significantly across different attack types and approaches. The Benchmark approach performs well for Scanning and DDoS attacks, achieving F1-Scores of 0.9890 and 0.9764, respectively. However, it struggles with DoS and MitM, where the F1-Scores drop to 0.6175 and 0.7239 (Fig. 5b). The $SPECTRA$ methodology substantially enhances the classification performance. $SPECTRA(\mu_1, \mu_2, \mu_3, \mu_4)$ achieves an F1-Score of 0.9944 for DDoS and 0.9650 for Ransomware, demonstrating its effectiveness in detecting complex attacks. The best performance is observed with $SPECTRA(\mu_1, \mu_2, \mu_3, \mu_4, \mu_5)$, which achieves an F1-Score of 1.0 for DDoS and 0.9889 for Ransomware (Fig. 5b), further validating the advantage of incorporating all spectral metrics. Balanced Accuracy results reinforce these findings. The Benchmark approach records a Balanced Accuracy of 0.8290 for Ransomware, while $SPECTRA(\mu_1, \mu_2, \mu_3, \mu_4, \mu_5)$ raises this to 0.9936 (Fig. 6b). MCC values show a similar trend, where the Benchmark approach achieves only 0.8193 for Ransomware, whereas $SPECTRA$ using all five spectral metrics reaches 0.9932 (Fig. 7b). These results confirm that the combination of all spectral metrics significantly improves the classifier's ability to accurately predict and distinguish between various attack types.

Key Findings: Across both datasets, $SPECTRA$ with all five spectral metrics ($\mu_1, \mu_2, \mu_3, \mu_4, \mu_5$) including *BiFlowness* consistently outperforms other approaches. It provides the highest F1-Scores, Balanced Accuracy, and MCC for multiple attack types, demonstrating that the integration of these metrics enhances classification accuracy and robustness. These findings validate the importance of spectral metrics with the combination of *BiFlowness* metric in refining network attack detection, particularly in distinguishing complex attack behaviors.

8 Conclusion

This work presents *SPECTRA*, a spectral graph analysis methodology tailored for monitoring and detecting anomalies in dynamic network environments, with a particular focus on medical IoT systems. By leveraging five spectral metrics, including a powerful *BiFlowness* metric derived from Singular Value Decomposition (SVD), *SPECTRA* effectively captures the flow and connectivity dynamics within hybrid graph topologies that combine star and bipartite structures. Through extensive evaluations on IoT datasets, the proposed approach demonstrates its capability to detect complex cyberattacks, such as multistep intrusions, with

good accuracy compared to other approaches methods. The ability to monitor evolving network structures in real time makes *SPECTRA* a good tool for enhancing security in sensitive and interconnected medical devices. Future research will explore the scalability of this approach to larger and more diverse datasets, as well as its applications.

References

- [AABM22] Teresa Alsinet, Josep Argelich, Ramón Béjar, and Santi Martínez. Approximate and optimal solutions for the bipartite polarization problem. In *Artificial Intelligence Research and Development*, pages 17–24. IOS Press, 2022.
- [Abd07] Hervé Abdi. Singular value decomposition (svd) and generalized singular value decomposition. *Encyclopedia of measurement and statistics*, 907(912):44, 2007.
- [Bal22] Gianmarco Baldini. Detection of cybersecurity spoofing attacks in vehicular networks with recurrence quantification analysis. *Computer Communications*, 191:486–499, 2022.
- [BC⁺06] Steve Butler, Fan Chung, et al. Spectral graph theory. *Handbook of linear algebra*, page 47, 2006.
- [BH21] Benjamin Bowman and H Howie Huang. Towards next-generation cybersecurity with graph ai. *ACM SIGOPS Operating Systems Review*, 55(1):61–67, 2021.
- [BJ09] Frank Bauer and Jürgen Jost. Bipartite and neighborhood graphs and the spectrum of the normalized graph laplacian. *arXiv preprint arXiv:0910.3118*, 2009.
- [BY06] Cemal Cagatay Bilgin and Bülent Yener. Dynamic network evolution: Models, clustering, anomaly detection. *IEEE Networks*, 1, 2006.
- [CCH16] Pin-Yu Chen, Sutanay Choudhury, and Alfred O Hero. Multi-centrality graph spectral decompositions and their application to cyber intrusion detection. In *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4553–4557. IEEE, 2016.
- [CCL⁺21] Lei Cai, Zhengzhang Chen, Chen Luo, Jiaping Gui, Jingchao Ni, Ding Li, and Haifeng Chen. Structural temporal graph neural networks for anomaly detection in dynamic graphs. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, pages 3747–3756, 2021.
- [CG16] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794, 2016.
- [CH06] Yu Chen and Kai Hwang. Collaborative detection and filtering of shrew ddos attacks using spectral analysis. *Journal of Parallel and Distributed Computing*, 66(9):1137–1151, 2006.
- [Chu97] Fan RK Chung. *Spectral graph theory*, volume 92. American Mathematical Soc., 1997.
- [CKT02] Chen-Mou Cheng, HT Kung, and Koan-Sin Tan. Use of spectral analysis in defense against dos attacks. In *Global Telecommunications Conference, 2002. GLOBECOM’02. IEEE*, volume 3, pages 2143–2148. IEEE, 2002.
- [CR23] José E Chacón and Ana I Rastrojo. Minimum adjusted rand index for two clusterings of a given size. *Advances in Data Analysis and Classification*, 17(1):125–133, 2023.
- [Dax13] Achiya Dax. From eigenvalues to singular values: a review. *Advances in Pure Mathematics*, 2013, 2013.
- [dLdRVDH14] Siemon de Lange, Marcel de Reus, and Martijn Van Den Heuvel. The laplacian spectrum of neural networks. *Frontiers in computational neuroscience*, 7:189, 2014.
- [DM17] Rishabh Das and Thomas H Morris. Machine learning and cyber security. In *2017 international conference on computer, electrical & communication engineering (ICCECE)*, pages 1–7. IEEE, 2017.
- [DMM22] Sagnik Dutta, Nilava Metya, and Sagnik Mukherjee. Fiedler vector method. 2022.
- [FHWL23] Si-Guo Fang, Dong Huang, Chang-Dong Wang, and Jian-Huang Lai. One-step bipartite graph cut: A normalized formulation and its application to scalable subspace clustering. *arXiv preprint arXiv:2305.07386*, 2023.
- [For07] Behrouz A Forouzan. *Data communications and networking*. Huga Media, 2007.
- [GL16] Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 855–864, 2016.
- [HGP20] Ayush Hariharan, Ankit Gupta, and Trisha Pal. Camlpad: Cybersecurity autonomous machine learning platform for anomaly detection. In *Future of Information and Communication Conference*, pages 705–720. Springer, 2020.
- [HRD16] Nick Heard and Patrick Rubin-Delanchy. Network-wide anomaly detection via the dirichlet process. In *2016 IEEE Conference on Intelligence and Security Informatics (ISI)*, pages 220–224. IEEE, 2016.
- [JBP24] Majed Jaber, Nicolas Boutry, and Pierre Parrend. Graph-based spectral analysis for detecting cyber attacks. In *Proceedings of the 19th International Conference on Availability, Reliability and Security*, pages 1–14, 2024.

- [JWT⁺19] Aimin Jiang, Jiaan Wan, Yibin Tang, Beilu Ni, and Yanping Zhu. Admm-based bipartite graph approximation. In *ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5421–5425. IEEE, 2019.
- [KMST19] Nickolaos Koroniotis, Nour Moustafa, Elena Sitnikova, and Benjamin Turnbull. Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset. *Future Generation Computer Systems*, 100:779–796, 2019.
- [Kun15] Jérôme Kunegis. Exploiting the structure of bipartite graphs for algebraic and spectral graph theory applications. *Internet Mathematics*, 11(3):201–321, 2015.
- [KW16] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.
- [LCQ⁺21] Yongsheng Liu, Wenyu Chen, Hong Qu, SM Hasan Mahmud, and Kebin Miao. Weakly supervised image classification and pointwise localization with graph convolutional networks. *Pattern Recognition*, 109:107596, 2021.
- [LL23] Lan Lin and Yixun Lin. Graph bipartization problem with applications to via minimization in vlsi design. *International Journal of Foundations of Computer Science*, 34(04):347–361, 2023.
- [Lóp07] Ricardo A López. Computer networks. a top-down approach featuring internet. *Journal of Computer Science and Technology*, 7(2):194, 2007.
- [LZL⁺23] Junhao Li, Jiacheng Zhou, Lei Li, Baowei Cheng, Shengfei Li, and Wenhao He. Robust detection for adversarial attacks-based on a graph neural network for web application security protection system. In *Proceedings of the 4th International Conference on Artificial Intelligence and Computer Engineering*, pages 927–932, 2023.
- [Mer94] Russell Merris. Laplacian matrices of graphs: a survey. *Linear algebra and its applications*, 197:143–176, 1994.
- [MW19] Subhadeep Mukhopadhyay and Kaijun Wang. Spectral graph analysis: A unified explanation and modern perspectives. *arXiv preprint arXiv:1901.07090*, 2019.
- [NA18] Jordan Noble and Niall Adams. Real-time dynamic network anomaly detection. *IEEE Intelligent Systems*, 33(2):5–18, 2018.
- [PARS14] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 701–710, 2014.
- [PMRC21] Sergio Iglesias Pérez, Santiago Moral-Rubio, and Regino Criado. A new approach to combine multiplex networks and time series attributes: Building intrusion detection systems (ids) in cybersecurity. *Chaos, Solitons & Fractals*, 150:111143, 2021.
- [SGB22] Debapriya Sengupta, Phalguni Gupta, and Arindam Biswas. A survey on mutual information based medical image registration algorithms. *Neurocomputing*, 486:174–188, 2022.
- [SLV⁺20] Kamran Shaukat, Suhuai Luo, Vijay Varadharajan, Ibrahim A Hameed, Shan Chen, Dongxi Liu, and Jiaming Li. Performance comparison and current challenges of using machine learning techniques in cybersecurity. *Energies*, 13(10):2509, 2020.
- [Sta03] William Stallings. *Network security essentials: Applications and standards*, 4/e. Pearson Education India, 2003.
- [TBSK18] Andrew Tomlinson, Jeremy Bryans, Siraj Ahmed Shaikh, and Harsha Kumara Kalutarage. Detection of automotive can cyber-attacks by identifying packet timing anomalies in time windows. In *2018 48th Annual IEEE/IFIP international conference on dependable systems and networks workshops (DSN-W)*, pages 231–238. IEEE, 2018.
- [VCC⁺17] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.
- [YNY⁺18] Naganand Yadati, Madhav Nimishakavi, Prateek Yadav, Anand Louis, and Partha Talukdar. HypergcN: Hypergraph convolutional networks for semi-supervised classification. *arXiv preprint arXiv:1809.02589*, 22, 2018.
- [YWB10] Xiaowei Ying, Xintao Wu, and Daniel Barbara. Spectrum based fraud detection in social networks. In *Proceedings of the 17th ACM conference on Computer and communications security*, pages 747–749, 2010.
- [ZCO17] Jin Zeng, Gene Cheung, and Antonio Ortega. Bipartite approximation for graph wavelet signal decomposition. *IEEE Transactions on Signal Processing*, 65(20):5466–5480, 2017.
- [ZHD⁺01] Hongyuan Zha, Xiaofeng He, Chris Ding, Horst Simon, and Ming Gu. Bipartite graph partitioning and data clustering. In *Proceedings of the tenth international conference on Information and knowledge management*, pages 25–32, 2001.